

DSA APPROACH HANDBOOK

THINK IN PATTERNS

Recognize → Choose → Justify →
Implement → Analyze

A practical interview-focused guide to understanding WHY an algorithm is chosen.

The goal is not to memorize 500 solutions. The goal is to look at a new problem and know what tool the problem is quietly asking for.

01 RECOGNIZE Find the pattern	02 CHOOSE Pick the tool	03 JUSTIFY Explain why
04 ANALYZE Time + space	05 PRACTICE Solve variations	06 INTERVIEW Communicate clearly

00 The Universal DSA Method

Use this before writing code. Every. Single. Time.

01 Understand

Input • output • constraints • what exactly is being asked

02 Brute Force

What would I do without optimization?

03 Find Bottleneck

What repeated work makes it slow?

04 Observe

Sorted? contiguous? monotonic? repeated? connected? dependent?

05 Choose

Data structure + algorithm/pattern

06 Justify

Why can I safely make these moves?

07 Analyze

Time + auxiliary space

08 Test

Edge cases + dry run

INTERVIEW SENTENCE

"The brute-force approach is ___. Its bottleneck is ___. I noticed ___. Therefore I can use ___, which reduces the complexity from ___ to ___."

01 HashMap & HashSet

RECOGNIZE THIS PATTERN

Fast existence check, frequency count, value→index, or key→value lookup.

WHY	TIME	SPACE
Hashing avoids repeated linear searches; average lookup is $O(1)$.	Average $O(1)$ per operation	$O(n)$

KEY THOUGHT

HashSet asks: 'Have I seen this?' HashMap asks: 'What information belongs to this key?'

PRACTICE PROBLEMS

1. Two Sum
2. Contains Duplicate
3. Valid Anagram
4. Group Anagrams
5. Longest Consecutive Sequence
6. Subarray Sum Equals K

02 Two Pointers

RECOGNIZE THIS PATTERN

Two positions can move through the data while eliminating many candidates, often with sorted arrays or opposite ends.

WHY	TIME	SPACE
A useful property tells you which pointer can safely move, replacing many pair checks.	Usually $O(n)$	Usually $O(1)$

KEY THOUGHT

left + right. The important part is not the two pointers, but WHY one pointer can move.

PRACTICE PROBLEMS

1. Valid Palindrome
2. Two Sum II
3. 3Sum
4. Container With Most Water
5. Remove Duplicates from Sorted Array
6. Trapping Rain Water

03 Sliding Window

RECOGNIZE THIS PATTERN

Contiguous subarray/substring with a condition, fixed size, or optimization.

WHY	TIME	SPACE
Keep one window and update it incrementally instead of recalculating every range.	Usually $O(n)$	$O(1)$ to $O(k)$

KEY THOUGHT

[left ... right] → expand right → if invalid, shrink left → record answer.

PRACTICE PROBLEMS

1. Maximum Sum Subarray of Size K
2. Longest Substring Without Repeating Characters
3. Minimum Window Substring
4. Longest Repeating Character Replacement
5. Permutation in String
6. Fruit Into Baskets

04 Prefix Sum

Make repeated range sums cheap.

THE FORMULA

Array: [2, 4, 1, 5, 3]

Prefix: [2, 6, 7, 12, 15]

Meaning: $\text{prefix}[i] = \text{sum from index 0 to } i$.

Range: $\text{sum}(l, r) = \text{prefix}[r] - \text{prefix}[l-1]$.

Why: $\text{prefix}[r]$ contains 0..r. Subtract $\text{prefix}[l-1]$, which contains 0..l-1. What remains is exactly l..r.

Example: $\text{sum}(2, 4) = 15 - 6 = 9$.

If $l=0$: $\text{sum}(0, r) = \text{prefix}[r]$.

Complexity: Build $O(n)$, each query $O(1)$, space $O(n)$.

1. Range Sum Query
2. Subarray Sum Equals K
3. Find Pivot Index
4. Continuous Subarray Sum

05 Sorting as a Tool

RECOGNIZE THIS PATTERN

Data is unsorted and ordering could expose a useful structure.

WHY	TIME	SPACE
Sorted data can unlock two pointers, binary search, greedy choices, duplicate handling, and interval merging.	Usually $O(n \log n)$	Depends

KEY THOUGHT

Ask: 'Would sorted order make the next decision obvious?'

PRACTICE PROBLEMS

1. 3Sum
2. Merge Intervals
3. Meeting Rooms
4. Non-overlapping Intervals
5. Assign Cookies

06 Binary Search

Normal search and the much more important 'search the answer' pattern.

NORMAL BINARY SEARCH

Trigger: Sorted search space.

Why: Compare with mid and discard half.

Time: $O(\log n)$. Space: $O(1)$.

BINARY SEARCH ON ANSWER

Trigger: 'Minimum X that works' or 'Maximum X that works.'

Key: The answer space is monotonic: FALSE FALSE FALSE TRUE TRUE TRUE.

Example: If capacity 10 can ship all packages, capacity 11+ can also work.

Process: Test mid $\rightarrow$ if it works, search smaller; if it fails, search larger.

Complexity: $O(\log(\text{answer range}) \times \text{check-cost})$.

1. Search Insert Position
2. First and Last Position
3. Search in Rotated Sorted Array
4. Find Minimum in Rotated Sorted Array
5. Koko Eating Bananas
6. Capacity to Ship Packages
7. Split Array Largest Sum
8. Minimum Days to Make M Bouquets

07 Stack

RECOGNIZE THIS PATTERN

Latest unresolved item must be handled first. LIFO.

WHY	TIME	SPACE
The top is exactly the newest unresolved item.	Push/pop/top $O(1)$	$O(n)$

KEY THOUGHT

Latest unresolved thing $\rightarrow$ Stack.

PRACTICE PROBLEMS

1. Valid Parentheses
2. Min Stack
3. Evaluate Reverse Polish Notation
4. Basic Calculator
5. Remove Adjacent Duplicates
6. Asteroid Collision

08 Monotonic Stack

RECOGNIZE THIS PATTERN

Need next/previous greater or smaller element, or nearest greater/smaller information.

WHY	TIME	SPACE
Pop elements that can no longer become answers. Every element is pushed and popped at most once.	$O(n)$	$O(n)$

KEY THOUGHT

Maintain increasing/decreasing order. The question decides which monotonic direction.

PRACTICE PROBLEMS

1. Next Greater Element I
2. Next Greater Element II
3. Daily Temperatures
4. Stock Span
5. Previous Smaller Element
6. Largest Rectangle in Histogram
7. Sum of Subarray Minimums

09 Queue

RECOGNIZE THIS PATTERN

First-in-first-out processing.

WHY	TIME	SPACE
The earliest inserted item must be processed first.	Enqueue/dequeue $O(1)$	$O(n)$

KEY THOUGHT

FIFO → Queue. Major DSA use: BFS.

PRACTICE PROBLEMS

1. Binary Tree Level Order Traversal
2. Rotting Oranges
3. Number of Islands (BFS)
4. Design Circular Queue
5. Scheduling Simulation

10 Linked List

RECOGNIZE THIS PATTERN

Nodes connected by links; efficient insertion/deletion by changing links.

WHY	TIME	SPACE
No shifting of later elements, but random access is $O(n)$.	Search/access $O(n)$; known-position insert/delete $O(1)$	Usually $O(1)$ extra

KEY THOUGHT

Reverse → prev/current/next. Middle/Cycle → slow/fast. Merge → two pointers.

PRACTICE PROBLEMS

1. Reverse Linked List
2. Middle of Linked List

3. Linked List Cycle
4. Remove Nth Node From End
5. Merge Two Sorted Lists
6. Intersection of Two Linked Lists
7. Palindrome Linked List

11 Recursion

RECOGNIZE THIS PATTERN

Same problem can be expressed on a smaller input.

WHY	TIME	SPACE
Solve smaller instances until a base case.	Depends on recurrence	Usually $O(\text{depth})$ stack

KEY THOUGHT

Base case + smaller problem + progress toward base case.

PRACTICE PROBLEMS

1. Factorial
2. Fibonacci
3. Tree Traversals
4. Merge Sort
5. Power(x,n)
6. Generate Parentheses

12 Backtracking

RECOGNIZE THIS PATTERN

Need all possibilities or constrained search.

WHY	TIME	SPACE
Choose one option, explore it, undo it, then try the next option.	Often $O(2^n)$ or $O(n!)$	Usually recursion depth

KEY THOUGHT

CHOOSE → EXPLORE → UNDO.

PRACTICE PROBLEMS

1. Subsets
2. Permutations
3. Combinations
4. Combination Sum
5. N-Queens
6. Sudoku Solver
7. Word Search

13 Trees

RECOGNIZE THIS PATTERN

Parent-child relationships.

WHY	TIME	SPACE
DFS explores branches; BFS explores levels.	Traversal $O(n)$	DFS $O(h)$, BFS $O(w)$

KEY THOUGHT

Preorder = parent first. Inorder = sorted order only for BST. Postorder = children first. Level order = BFS.

PRACTICE PROBLEMS

1. Maximum Depth
2. Invert Binary Tree
3. Diameter of Binary Tree
4. Balanced Binary Tree
5. Path Sum
6. Lowest Common Ancestor
7. Level Order Traversal

TREE TRAVERSAL MAP

Preorder: Root → Left → Right

Inorder: Left → Root → Right

Postorder: Left → Right → Root

Level Order: BFS + Queue

14 Binary Search Tree (BST)

RECOGNIZE THIS PATTERN

Tree obeys $\text{left} < \text{root} < \text{right}$.

WHY	TIME	SPACE
Ordering lets search move toward one side; inorder traversal gives sorted values.	Average $O(\log n)$, worst $O(n)$	$O(h)$

KEY THOUGHT

BST property + inorder = sorted order.

PRACTICE PROBLEMS

1. Search in BST
2. Validate BST
3. Kth Smallest in BST
4. Lowest Common Ancestor of BST
5. Convert Sorted Array to BST
6. Inorder Successor

15 Heap / Priority Queue

RECOGNIZE THIS PATTERN

Repeatedly need current minimum/maximum while data changes.

WHY	TIME	SPACE
Get the extreme value without fully sorting everything.	Push/pop $O(\log n)$, top $O(1)$, build $O(n)$	$O(n)$, or $O(k)$

KEY THOUGHT

Only need best K ? Keep heap size $K \rightarrow$ often $O(n \log k)$.

PRACTICE PROBLEMS

1. Kth Largest
2. Top K Frequent Elements
3. K Closest Points
4. Merge K Sorted Lists
5. Find Median from Data Stream
6. Task Scheduler

16 Greedy

Local choice can be globally safe, but it needs justification.

UNDERSTAND GREEDY

Meaning: Make the best local choice now and don't undo it.

Why use it: Some problems have a proof that the local choice can appear in an optimal solution.

Danger: A choice that merely 'looks best' is not enough.

Common proof: Exchange argument. Show an optimal solution can be changed to use the greedy choice without getting worse.

Typical complexity: Often $O(n \log n)$ because sorting comes first.

1. Activity Selection
2. Fractional Knapsack
3. Jump Game
4. Gas Station
5. Partition Labels
6. Non-overlapping Intervals
7. Minimum Arrows to Burst Balloons

17 Dynamic Programming

DP is about states, transitions, and reusing answers.

DP CHECKLIST

1. State: What exactly does $dp[i]$ or $dp[i][j]$ mean?
2. Last decision: What choices lead into this state?

3. Transition: How do smaller states build the current one?
 4. Base case: Smallest known answer.
 5. Store: Memoization or tabulation.
- Core trigger: Overlapping subproblems + optimal substructure.

18 1D DP

RECOGNIZE THIS PATTERN

One variable defines the state.

WHY	TIME	SPACE
Previous answers are reused instead of recomputed.	Often $O(n)$	Often $O(n)$, sometimes $O(1)$

KEY THOUGHT

Climbing Stairs: last move is 1 or 2 $\rightarrow dp[n] = dp[n-1] + dp[n-2]$.

PRACTICE PROBLEMS

1. Climbing Stairs
2. Min Cost Climbing Stairs
3. House Robber
4. House Robber II
5. Coin Change
6. Decode Ways
7. Maximum Product Subarray

19 2D DP

RECOGNIZE THIS PATTERN

Two variables define the state.

WHY	TIME	SPACE
The answer depends on two positions/parameters.	Often $O(nm)$	Often $O(nm)$, sometimes $O(m)$

KEY THOUGHT

LCS: $dp[i][j]$ = answer for first i chars of A and first j chars of B.

PRACTICE PROBLEMS

1. Unique Paths
2. Minimum Path Sum
3. Longest Common Subsequence
4. Edit Distance
5. Longest Palindromic Subsequence
6. 0/1 Knapsack

20 Knapsack Pattern

The classic 'Take or Don't Take' decision.

CORE IDEA

Trigger: Items + capacity/target + choice.

Decision: Take current item OR skip it.

0/1 Knapsack: Each item at most once.

Unbounded: Item can be reused.

Why DP: Different choices reach the same remaining target/capacity, creating repeated subproblems.

Problems: 0/1 Knapsack, Subset Sum, Partition Equal Subset Sum, Target Sum, Coin Change, Coin Change II.

21 Graphs

First classify the graph. Then choose the algorithm.

GRAPH CHECKLIST

1. Directed or undirected?
2. Weighted or unweighted?
3. Connected or multiple components?
4. Cyclic or acyclic?
5. Asked for reachability, shortest path, dependency order, components, or minimum connection?

22 BFS

RECOGNIZE THIS PATTERN

Unweighted shortest path or level-by-level exploration.

WHY	TIME	SPACE
BFS visits distance 0, then 1, then 2, so first reach gives minimum edge distance.	$O(V+E)$	$O(V)$

KEY THOUGHT

Queue + visited + levels/distance.

PRACTICE PROBLEMS

1. Number of Islands
2. Rotting Oranges
3. Shortest Path in Binary Matrix
4. Word Ladder
5. 01 Matrix
6. Binary Tree Level Order

23 DFS

RECOGNIZE THIS PATTERN

Deep exploration, connected components, or recursive graph processing.

WHY	TIME	SPACE
Follow one route deeply, then return and explore the next.	$O(V+E)$	$O(V)$

KEY THOUGHT

Recursion/stack + visited.

PRACTICE PROBLEMS

1. Number of Islands
2. Flood Fill

3. Clone Graph
4. Path Exists
5. Connected Components
6. Cycle Detection
7. Surrounded Regions

24 Topological Sort

RECOGNIZE THIS PATTERN

Directed dependencies: A must happen before B.

WHY	TIME	SPACE
Creates an ordering respecting prerequisites when the graph is a DAG.	$O(V+E)$	$O(V)$

KEY THOUGHT

Kahn: indegree + queue. If processed nodes $< V$, there is a cycle.

PRACTICE PROBLEMS

1. Course Schedule
2. Course Schedule II
3. Alien Dictionary
4. Parallel Courses
5. Build Order

25 Union Find / DSU

RECOGNIZE THIS PATTERN

Repeatedly ask whether two nodes are in the same component or merge components.

WHY	TIME	SPACE
Union merges groups; Find identifies a group's representative.	Nearly $O(1)$ amortized	$O(V)$

KEY THOUGHT

$\text{find}(x) = \text{representative}$; $\text{union}(a,b) = \text{merge groups}$.

PRACTICE PROBLEMS

1. Number of Provinces
2. Redundant Connection
3. Accounts Merge
4. Kruskal MST
5. Connected Components
6. Network Connectivity

26 Shortest Path

Choose based on the graph's edge properties.

GRAPH	USE	WHY
Unweighted	BFS	Fewest edges first
Positive weights	Dijkstra	Smallest tentative distance first
Negative edges	Bellman-Ford	Supports negative edges
All pairs	Floyd-Warshall	DP over intermediate vertices
DAG	Topological DP	Process in dependency order

REMEMBER

Do not automatically use Dijkstra for every shortest-path problem. Humans have invented enough algorithms that apparently you must first inspect the weights.

27 Trie

RECOGNIZE THIS PATTERN

Many words share prefixes; need prefix/dictionary operations.

WHY	TIME	SPACE
Characters are stored along paths, making prefix search natural.	$O(L)$ per insert/search	$O(\text{total stored characters})$

KEY THOUGHT

Character-by-character tree → prefix problems.

PRACTICE PROBLEMS

1. Implement Trie
2. Design Add and Search Words
3. Word Search II
4. Replace Words
5. Autocomplete

28 Bit Manipulation

RECOGNIZE THIS PATTERN

Binary representation, XOR, powers of two, masks, or constant-space tricks.

WHY	TIME	SPACE
Bit properties can solve some tasks with tiny memory usage.	Usually $O(1)$ per bit operation	Often $O(1)$

KEY THOUGHT

$x \& 1 \rightarrow \text{odd/even}$ | $x < 1 \rightarrow x \times 2$ if safe | $x > 1 \rightarrow \text{right shift}$ | $x \wedge x = 0$ | $x \wedge 0 = x$.

PRACTICE PROBLEMS

1. Single Number
2. Missing Number

3. Number of 1 Bits
4. Counting Bits
5. Power of Two
6. Reverse Bits
7. Bitmask Subsets

29 Intervals

RECOGNIZE THIS PATTERN

Ranges [start,end] with overlap, merging, scheduling, or gaps.

WHY	TIME	SPACE
Sorting makes neighboring interval relationships predictable.	Usually $O(n \log n)$	Usually $O(n)$

KEY THOUGHT

Sort by start/end → scan once.

PRACTICE PROBLEMS

1. Merge Intervals
2. Insert Interval
3. Non-overlapping Intervals
4. Meeting Rooms
5. Meeting Rooms II
6. Minimum Arrows

30 Kadane's Algorithm

RECOGNIZE THIS PATTERN

Maximum sum of a contiguous subarray.

WHY	TIME	SPACE
At each position, extend the current subarray or start fresh.	$O(n)$	$O(1)$

KEY THOUGHT

$\text{current} = \max(\text{nums}[i], \text{current} + \text{nums}[i]).$

PRACTICE PROBLEMS

1. Maximum Subarray
2. Maximum Sum Circular Subarray

31 Extra High-Value Patterns

31 Extra High-Value Patterns

31A Divide & Conquer

RECOGNIZE THIS PATTERN

Split into smaller independent problems and combine their answers.

WHY	TIME	SPACE
Solve smaller parts, then combine.	Depends on recurrence	Usually recursion depth

KEY THOUGHT

Divide → Solve → Combine.

PRACTICE PROBLEMS

1. Merge Sort
2. Quick Sort
3. Maximum Subarray
4. Closest Pair of Points

31B Matrix / Grid as Graph

RECOGNIZE THIS PATTERN

2D cells have neighbor relationships.

WHY	TIME	SPACE
Treat each cell as a graph node.	Usually $O(\text{rows} \times \text{cols})$	$O(\text{rows} \times \text{cols})$

KEY THOUGHT

Directions + bounds + visited.

PRACTICE PROBLEMS

1. Number of Islands
2. Flood Fill
3. Rotting Oranges
4. Surrounded Regions
5. Pacific Atlantic Water Flow
6. Word Search

31C Meet in the Middle

RECOGNIZE THIS PATTERN

2^n is too large, but splitting into two halves is manageable.

WHY	TIME	SPACE
-----	------	-------

Enumerate both halves and combine their results.	Often $O(2^{(n/2)})$	Often $O(2^{(n/2)})$
--	----------------------	----------------------

KEY THOUGHT

Split → solve halves → combine.

PRACTICE PROBLEMS

1. Closest Subsequence Sum
2. Subset Sum variants
3. Small-n partition problems

31D State-Space BFS/DFS

RECOGNIZE THIS PATTERN

Position alone is not enough; extra state changes the answer.

WHY	TIME	SPACE
Treat position + extra information as the graph state.	Depends on number of states	Depends on states

KEY THOUGHT

Ask: what else must I remember?

PRACTICE PROBLEMS

1. Word Ladder
2. Shortest Path with Keys
3. Grid with remaining resources

32. MASTER PRACTICE ROADMAP

A. ARRAYS / STRINGS

1. Two Sum
2. Best Time to Buy and Sell Stock
3. Contains Duplicate
4. Product of Array Except Self
5. Maximum Subarray
6. Maximum Product Subarray
7. Move Zeroes
8. Rotate Array
9. Valid Anagram
10. Group Anagrams
11. Valid Palindrome
12. Longest Palindromic Substring

B. TWO POINTERS

1. Valid Palindrome
2. Two Sum II
3. 3Sum
4. Container With Most Water

5. Remove Duplicates from Sorted Array
6. Trapping Rain Water

C. SLIDING WINDOW

1. Maximum Sum Subarray of Size K
2. Longest Substring Without Repeating Characters
3. Longest Repeating Character Replacement
4. Minimum Window Substring
5. Permutation in String
6. Fruit Into Baskets

D. HASHING / PREFIX

1. Contains Duplicate
2. Two Sum
3. Valid Anagram
4. Group Anagrams
5. Longest Consecutive Sequence
6. Subarray Sum Equals K
7. Find Pivot Index
8. Continuous Subarray Sum

E. BINARY SEARCH

1. Binary Search
2. Search Insert Position
3. First and Last Position
4. Search in Rotated Sorted Array
5. Find Minimum in Rotated Sorted Array
6. Find Peak Element
7. Koko Eating Bananas
8. Capacity to Ship Packages
9. Split Array Largest Sum
10. Minimum Days to Make M Bouquets

F. STACK / MONOTONIC STACK

1. Valid Parentheses
2. Min Stack
3. Evaluate Reverse Polish Notation
4. Daily Temperatures
5. Next Greater Element I
6. Next Greater Element II
7. Stock Span
8. Largest Rectangle in Histogram
9. Sum of Subarray Minimums
10. Asteroid Collision

G. QUEUE / LINKED LIST

1. Design Circular Queue
2. Binary Tree Level Order Traversal
3. Reverse Linked List
4. Middle of the Linked List

5. Linked List Cycle
6. Remove Nth Node From End
7. Merge Two Sorted Lists
8. Intersection of Two Linked Lists
9. Palindrome Linked List
10. Reorder List

H. RECURSION / BACKTRACKING

1. Factorial
2. Fibonacci
3. Subsets
4. Permutations
5. Combinations
6. Combination Sum
7. Letter Combinations of a Phone Number
8. Generate Parentheses
9. N-Queens
10. Sudoku Solver
11. Word Search

I. TREES / BST

1. Maximum Depth of Binary Tree
2. Invert Binary Tree
3. Preorder Traversal
4. Inorder Traversal
5. Postorder Traversal
6. Level Order Traversal
7. Diameter of Binary Tree
8. Balanced Binary Tree
9. Lowest Common Ancestor
10. Validate Binary Search Tree
11. Kth Smallest Element in a BST
12. Search in a BST

J. HEAP / GREEDY / INTERVALS

1. Kth Largest Element
2. Top K Frequent Elements
3. K Closest Points to Origin
4. Merge K Sorted Lists
5. Find Median from Data Stream
6. Activity Selection
7. Jump Game
8. Gas Station
9. Merge Intervals
10. Insert Interval
11. Non-overlapping Intervals
12. Meeting Rooms II

K. DP

1. Climbing Stairs
2. Min Cost Climbing Stairs

3. House Robber
4. House Robber II
5. Coin Change
6. Coin Change II
7. Partition Equal Subset Sum
8. Target Sum
9. Unique Paths
10. Minimum Path Sum
11. Longest Common Subsequence
12. Edit Distance
13. Longest Increasing Subsequence
14. Longest Palindromic Subsequence
15. 0/1 Knapsack

L. GRAPHS

1. Number of Islands
2. Flood Fill
3. Clone Graph
4. Rotting Oranges
5. Course Schedule
6. Course Schedule II
7. Word Ladder
8. Shortest Path in Binary Matrix
9. Number of Provinces
10. Redundant Connection
11. Network Connectivity
12. Accounts Merge
13. Dijkstra
14. Bellman-Ford
15. Kruskal MST

M. TRIE / BITS

1. Implement Trie
2. Design Add and Search Words
3. Word Search II
4. Replace Words
5. Single Number
6. Missing Number
7. Number of 1 Bits
8. Counting Bits
9. Power of Two
10. Reverse Bits

33 One-Minute Pattern Recognition

01	Fast lookup / frequency	HashMap / HashSet
02	Sorted + pair / opposite ends	Two Pointers
03	Contiguous range	Sliding Window
04	Many range sums	Prefix Sum
05	Ordering simplifies decisions	Sorting
06	Sorted search	Binary Search
07	Minimum / maximum X that works	Binary Search on Answer
08	Latest unresolved item	Stack
09	Next / previous greater or smaller	Monotonic Stack
10	First-in-first-out	Queue
11	Parent-child relationship	Tree
12	Unweighted shortest path	BFS
13	Deep component exploration	DFS
14	Prerequisites / dependencies	Topological Sort
15	Same connected component?	DSU
16	Repeated min/max	Heap
17	All possible choices	Backtracking
18	Repeated subproblems + optimization	DP
19	Take / don't take	Knapsack DP
20	Prefix-heavy strings	Trie
21	Overlapping ranges	Intervals
22	Maximum contiguous sum	Kadane
23	XOR / binary / masks	Bit Manipulation

34 Interview Explanation Template

SAY THIS

1. Brute force: I could ___, but that takes ___.
2. Bottleneck: The expensive part is ___.
3. Observation: I noticed ___.
4. Choice: Because of that, I can use ___.
5. Why: This lets me ___ in ___ time.

6. Complexity: Overall time is ___ and space is ___.

7. Edge cases: duplicates, empty/single input, boundaries, negatives, overflow.

35 Final Rule

DON'T MEMORIZE THE SOLUTION. MEMORIZE THE REASON.

When you see a new problem, ask: What is expensive? → What property can I exploit? → What data structure makes that cheap? → What pattern fits?

Once you can answer those four questions, DSA becomes a decision-making process instead of a pile of unrelated tricks.